

Chapter 26

The Professional Toolkit

Git, GitHub and pull requests; the four undos; keeping secrets out of a repository; one automated check; a README a stranger can follow; Agile, Scrum, Kanban and Jira; and working with an AI assistant.

Draft chapter · v1

Contents

Chapter 26. The Professional Toolkit: Git, Agile, Documentation & AI Assistants

Why this matters

In plain English

26.1 The three places a file lives

26.2 Your first repository

26.3 Reading the history, and reading a change

26.4 What must never go into a repository

26.5 Undoing things: which of the four are you in?

26.6 Branches, and why they are not an advanced topic

26.7 GitHub, pushing, and the pull request

26.8 One automated check on every push

26.9 A repository a stranger can run

26.10 How the work is actually planned

26.11 Working with an AI assistant

Common mistakes and how to spot them

In the real world: the rule nobody could date

Tools

The project: one repository for everything you have built

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 26. The Professional Toolkit: Git, Agile, Documentation & AI Assistants

Part II — The Analyst

Chapter at a glance

You will learn to: explain what version control solves, and why "final_v3 REALLY_FINAL.xlsx" is a symptom · create a repository and record work in it with `git add`, `git commit`, and `git log` · read `git status` and `git diff` and say exactly what each line means · keep secrets and generated files out of a repository with `.gitignore` and a `.env` file · undo the four things that go wrong, and know which of the four you are in before you type anything · use a branch to try something without breaking what works · push to GitHub, open a pull request, and review someone else's · add one automated check that runs on every push · write a README that lets a stranger run your work · keep a SQL pattern library and documentation that people can still find in six months · work inside Scrum and Kanban, and read a Jira board without being told · use an AI assistant at work: what to delegate, how to check what it gives you, and what must never be pasted into one.

Before you start: Chapters 12 and 13 (the SQL you will be versioning), Chapters 17 and 18 (the Python scripts), and Chapter 20 (the automated report). Chapter 6 set up the tools, including Git. Nothing here needs a paid account.

Time needed: 6–8 hours, including the exercises and the project.

Tools: Git 2.43 or later, a free GitHub account, and a text editor. Everything in this chapter was run on Git 2.43.0 on Linux; the commands are identical on macOS and on Windows through Git Bash or WSL.

Practice data: none new. You will version the work you have already done in Part II: the queries from Chapters 12 and 13, the scripts from Chapters 17, 18, and 20. The worked example is one SQL file, built from nothing to a small shared repository.

Why this matters

Chapter 2 introduced Imran's Friday file: one spreadsheet, on one laptop, that only one person knew how to update, and that stopped when he took leave. Every chapter since has quietly added to the same pile. By now you have queries in Chapter 12, a pattern library in Chapter 13, workbooks in Chapters 10 and 11, cleaning scripts in Chapter 14, a report script in Chapter 20, and a dashboard in Chapter 16.

Where are they?

If the honest answer is "in a folder, some of them with a date in the name", then you have the Friday file problem at a larger scale. This chapter fixes it, and it fixes three other things at the same time, because they are the same problem wearing different clothes:

- **Nobody can run your work but you.** A repository with a README is the difference between a script and a thing your colleague can use on Monday.
- **Nobody can see how the team is doing.** Agile, and the board it runs on, is how data work gets planned in most companies you will join.
- **Nobody can check what an AI assistant produced.** You will use one. The skill is not prompting; it is knowing what to hand over and how to verify what comes back.

This is the chapter that turns a person who can analyze data into a person who can be hired to do it on a team. Twenty-one chapters of this book point here, which is a fair signal of how much of the day-to-day it covers.

In plain English

Version control is a laboratory notebook for files.

A scientist does not erase yesterday's page and write over it. They write today's entry underneath, dated and signed, with a note saying what changed and why. Every page is kept. If Thursday's experiment was wrong, you turn back to Wednesday and see exactly what was different. If two people work on the same project, each writes in their own notebook and they reconcile the pages afterward.

Git is that notebook for files. It does not save every keystroke. It saves **the snapshots you decide are worth keeping**, each with a note from you saying what changed. And because it keeps them all, you can go back, compare two days, or undo a change made three weeks ago without losing anything made since.

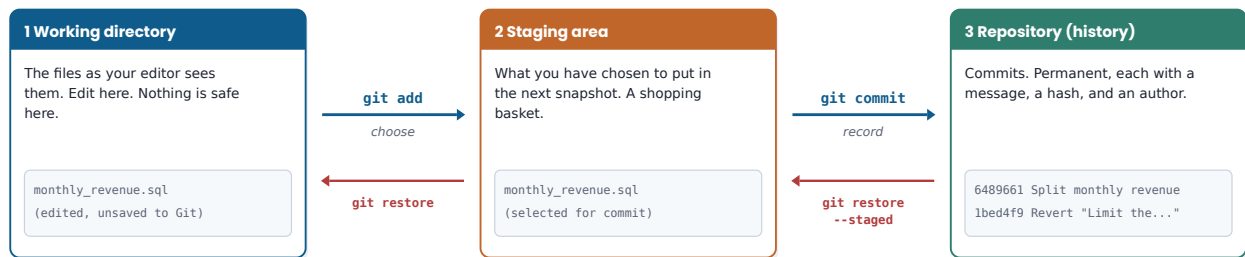
The reason people find it hard is not the idea. It is that Git keeps files in **three places at once**, and almost every confusing error message is Git telling you which of the three it is talking about.

26.1 The three places a file lives

Before any commands, the mental model. It is the whole of Git's difficulty and it takes two minutes.

The three places a file lives, and the commands that move it

Almost every confusing Git message is Git telling you which of these three it means.



And the three commands that only look:

<code>git status</code>	which files differ, and in which of the three
<code>git diff</code>	what changed in the working directory, line by line
<code>git log</code>	what is already in the history

Figure 26.1 – Every Git command you will use in this chapter moves a file between two of these, or shows you the difference between two of them.

Place	What it holds	How to see it
The working directory	the files as they are on your disk right now, including edits you have not recorded	open the file, or <code>git diff</code>
The staging area	the changes you have chosen to include in the <i>next</i> snapshot	<code>git diff --staged</code> , or <code>git status</code>
The repository	every snapshot you have ever recorded, with its note and its author	<code>git log</code>

`git add` moves a change from the working directory into the staging area. `git commit` turns everything in the staging area into a permanent snapshot in the repository.

Why staging exists at all, because this is the part beginners find fussy: it lets you record *some* of your changes. You fixed a query and, while you were in there, changed a comment in a different file. Those are two different things, and a history where each snapshot does one thing is a history you can read later. Staging is how you separate them.

26.2 Your first repository

The question: I have a SQL file I care about. How do I start keeping its history?

The plan, in words. Make a folder a repository. Ask Git what it can see. Choose the file for the next snapshot. Take the snapshot with a note. Then look at the history.

Start in a folder with one file in it, `monthly_revenue.sql`, the query from Chapter 13.

```
# terminal, in a new folder
$ git init
Initialized empty Git repository in /home/meera/riverstone-analysis/.git/

$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    monthly_revenue.sql

nothing added to commit but untracked files present (use "git add" to track)
```

How it works:

- **git init** creates a hidden folder called `.git` inside your current folder. That folder is the repository: the entire history lives there. Deleting it deletes the history and leaves your files alone.
- **git status** is the command you will type more than any other. It answers "what does Git see right now?" Read its three parts every time.
- **On branch main** names the line of history you are working on. Section 26.5 explains branches; for now, `main` is the only one.
- **No commits yet** means the repository is empty. You have a notebook with no pages.
- **Untracked files** means Git can see `monthly_revenue.sql` but is not watching it. Git never records a file until you tell it to, which is deliberate: your folder is full of things that should not be in the history.

Now choose the file and record it.

```
# terminal
$ git add monthly_revenue.sql

$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   monthly_revenue.sql

$ git commit -m "Add monthly revenue query"
[main (root-commit) e71cc3d] Add monthly revenue query
1 file changed, 6 insertions(+)
create mode 100644 monthly_revenue.sql
```

How it works:

- **git add monthly_revenue.sql** moves that file's contents into the staging area. It prints nothing, which is Git's way of saying it worked.
- The heading changed from *Untracked files* to **Changes to be committed**. That is the staging area, and the file is now in it.

- `git commit` takes everything staged and writes it into the repository as one snapshot, called a **commit**.
- `-m "Add monthly revenue query"` attaches the note. Without `-m`, Git opens a text editor and waits, which is the most common way a beginner gets stuck in a terminal they cannot leave.
- `[main (root-commit) e71cc3d]` is the confirmation: the branch, the fact that this is the first commit in the repository, and the commit's short **hash**, a unique name Git generates from the content. Yours will differ, because the hash includes the author and the time.
- `1 file changed, 6 insertions(+)` counts lines, not files. Six lines were added.

The settings on `git commit`

Setting	What it means in plain words	Value used here	What happens if you change it
<code>-m "..."</code>	the note attached to this snapshot	<code>"Add monthly revenue query"</code>	leave it out and Git opens an editor and waits for you to type the note there
<code>-a</code>	stage every already-tracked file that changed, then commit	not used here	saves a <code>git add</code> step, but silently includes edits you may not have meant to include. New files are still not added
<code>--amend</code>	replace the previous commit instead of adding a new one	not used here	fixes a bad message or a forgotten file. Never use it on a commit you have already pushed: section 26.4 explains why
<code>--no-verify</code>	skip any checks configured to run before committing	not used here	commits even when a check would have failed. Reach for it only when you know which check you are skipping and why

What a commit message should say

A commit message is written for the person who reads it in eight months, which is usually you.

Instead of	Write
<code>update</code>	<code>Fix month grouping: revenue was landing in the wrong month for orders on the 1st</code>
<code>fixes</code>	<code>Exclude cancelled orders from monthly revenue</code>
<code>asdf</code>	<code>anything</code>

The shape that works: a short line saying what changed, in the present tense, specific enough to be useful in a list of fifty. If the *why* cannot be read from the change itself, put it in a second paragraph after a blank line.

26.3 Reading the history, and reading a change

Two commands, and a demonstration of what one setting does.

The question: what has been recorded, and what exactly did I change since the last snapshot?

```
# terminal
$ git log --oneline
e71cc3d Add monthly revenue query
```

That is the whole history: one commit. Now make a change, and look at it before recording it. Add a line restricting the query to 2025 onward.

```
# terminal
$ git diff
diff --git a/monthly_revenue.sql b/monthly_revenue.sql
index 9a9cda2..391bab5 100644
--- a/monthly_revenue.sql
+++ b/monthly_revenue.sql
@@ -2,5 +2,6 @@
    SELECT date_trunc('month', order_date) AS mon,
           SUM(net_revenue)                AS revenue
    FROM   sales_lines
+WHERE   order_date >= DATE '2025-01-01'
    GROUP BY mon
    ORDER BY mon;
```

How it works:

- **git diff** shows what changed in the working directory that is **not yet staged**. This is the single most useful habit in this chapter: run it before every commit and read what you are about to record.
- **--- a/...** is the file before, **+++ b/...** is the file after.
- **@@ -2,5 +2,6 @@** says this block starts at line 2 and was 5 lines long before, and starts at line 2 and is 6 lines long now.
- A line starting with **+** was added. A line starting with **-** was removed. A line starting with a space is unchanged and is shown for context.
- **index 9a9cda2..391bab5** names the file's content before and after. You will rarely need it.

The related command is **git diff --staged**, which shows what is in the staging area waiting to be committed. **git diff** and **git diff --staged** answer two different questions, which is why a change can seem to vanish from **git diff** the moment you **git add** it.

A measured what-if: **--oneline**

Section 26.2's **git log --oneline** is a setting, so here is what it actually does. The same two commits, three ways.

```
# terminal
$ git log -n 2
commit 64896611c90a1fd237951846fe81b81d4fc49573
Author: Meera Iyer <meera@riverstone.example>
Date:   Mon Mar 2 10:02:00 2026 +0530

    Split monthly revenue by product category

commit 1bed4f99a50358b2e142cf5514e1bf48af0b77f9
Author: Meera Iyer <meera@riverstone.example>
Date:   Mon Mar 2 10:02:00 2026 +0530

    Revert "Limit the revenue query to 2025 onward"
```

```
# terminal
$ git log --oneline -n 2 --stat
6489661 Split monthly revenue by product category
monthly_revenue.sql | 5 +++-
1 file changed, 3 insertions(+), 2 deletions(-)
1bed4f9 Revert "Limit the revenue query to 2025 onward"
monthly_revenue.sql | 1 -
1 file changed, 1 deletion(-)
```

Setting	What it means in plain words	What happens if you change it
--oneline	one line per commit: short hash and message only	leave it out and you get the full hash, author, date, and message, about five lines per commit
-n 2	show only the two most recent commits	raise it to see more, leave it out to see all of them
--stat	add which files changed and how many lines	leave it out and you see the messages only. Useful when you are looking for "which commit touched this file"
--graph	draw the branch structure as lines down the left	on a history with branches this is how you see what merged into what. On a straight history it adds nothing

`git log --oneline --graph --stat -n 20` is a reasonable thing to alias to something short. You will type some version of it most days.

26.4 What must never go into a repository

A repository is public by default in your head, whatever its setting on GitHub. Two categories of file must stay out, and one of them will get someone fired eventually if this habit is not automatic.

Secrets. Passwords, API keys, connection strings, tokens. Chapter 20's report script needs a database password. Chapter 18's API example needs a token. These go in a file called `.env`, which stays on your machine, and the code reads them from there.

Generated output. Anything a script produces: exported CSVs, built workbooks, rendered charts, `__pycache__`. If it can be rebuilt by running the code, versioning it adds noise to every `git diff` and makes the repository large for no benefit.

The question: how do I tell Git to ignore these permanently?

Here is what Git sees when a `.env` and an `exports` folder exist and nothing has been told to ignore them.

```
# terminal
$ git status --short
?? .env
?? exports/
```

`git status --short` is the compact form: two columns of status codes and a filename. `??` means untracked. This is the moment where `git add .` would commit your database password.

Now add a file called `.gitignore` in the top of the repository:

```
# .gitignore
# Secrets: never commit these
.env

# Generated output: rebuildable, so not worth versioning
exports/
*.xlsx

# Local noise
.ipynb_checkpoints/
__pycache__/
```

```
# terminal
$ git status --short
?? .gitignore
```

How it works, line by line:

- A line beginning with `#` is a comment. Use them. A `.gitignore` with no explanation becomes a file nobody dares change.
- `.env` matches that exact file name anywhere in the repository.
- `exports/` with a trailing slash matches a *directory* and everything inside it. Without the slash it would also match a file named `exports`.
- `*.xlsx` matches by pattern: every file ending in `.xlsx`, in any folder.
- After adding it, `.env` and `exports/` are gone from `git status`. The only untracked file left is `.gitignore` itself, which **should** be committed, because everyone working on the repository needs the same rules.

To check what a rule is actually doing, ask Git rather than guessing:

```
# terminal
$ git check-ignore -v .env exports/jan.csv
.gitignore:2:.env    .env
.gitignore:5:exports/ exports/jan.csv
```

Each line says which file, at which line number of `.gitignore`, matched which path. If a file you expected to be ignored is not, this command tells you in one step whether the rule is wrong or whether the file was already tracked before the rule existed.

The trap that catches everyone once. `.gitignore` only affects files Git is **not already tracking**. If you committed `.env` last week and add the rule today, the file stays tracked and keeps being committed. You have to remove it from tracking explicitly with `git rm --cached .env`, commit that, and then change the password, because the old one is in the history and the history is what gets shared.

How the code reads a secret instead. Chapter 20's script does not contain the password. It reads it at runtime:

```
import os
password = os.environ["DB_PASSWORD"]
```

- `os.environ` is the set of environment variables the program was started with. The `.env` file is loaded into them, by your shell or by a small library, before the script runs.
- `["DB_PASSWORD"]` fails loudly with a `KeyError` if the variable is missing, which is what you want. A missing password should stop the script, not produce a confusing connection error twenty lines later.

Commit a `.env.example` instead, with the names and no values, so a colleague knows what they need to supply:

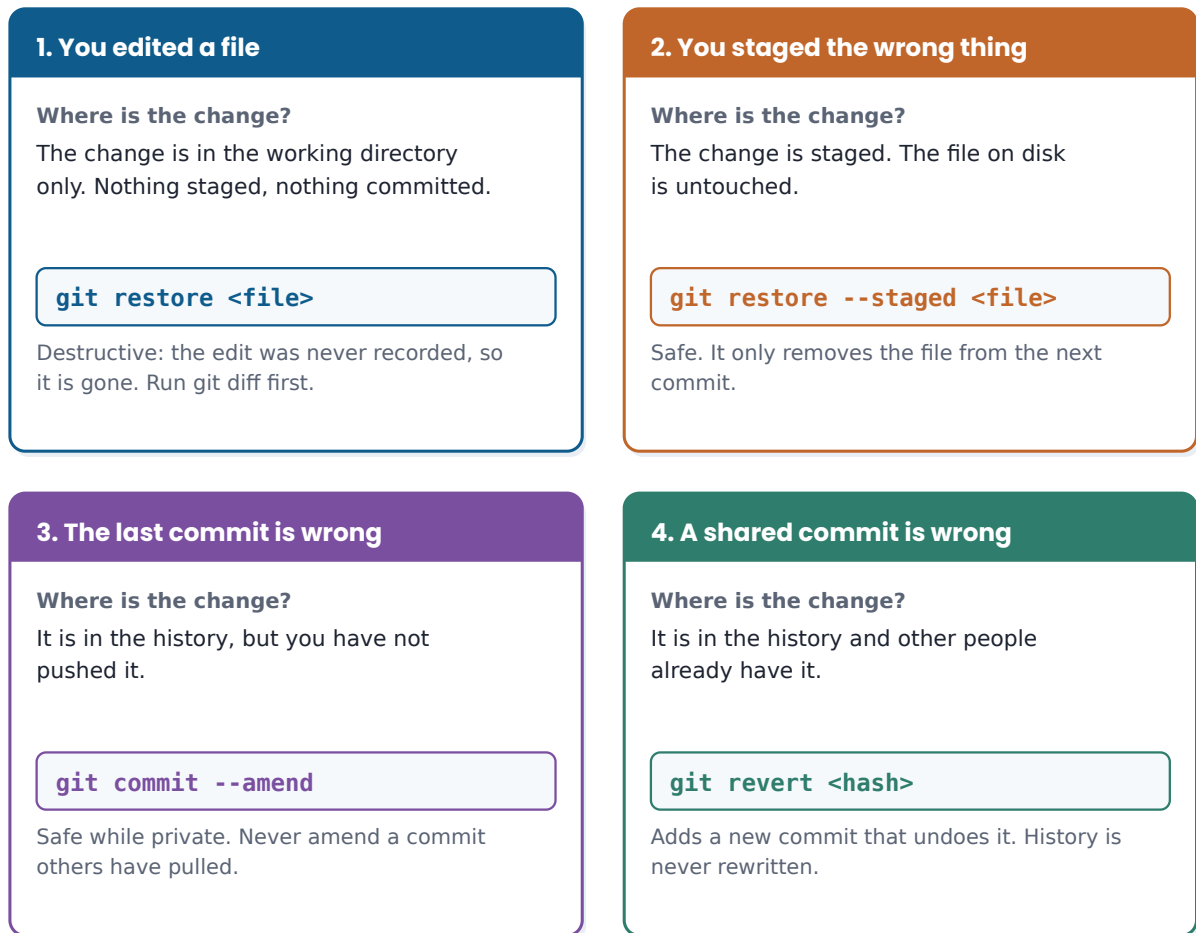
```
# .env.example
DB_PASSWORD=
DB_HOST=localhost
```

26.5 Undoing things: which of the four are you in?

Most Git fear is undo fear. There are four situations, and the first thing to do is work out which one you are in, because the command for each is different and using the wrong one is how people lose work.

Four things go wrong. Ask where the change is before asking how to undo it.

Choosing between these is the whole skill. Typing them is not.



git reset --hard is not on this list on purpose. It is the command that loses work, and the four above cover almost everything you will meet in your first two years.

Figure 26.2 – Ask where the change is before asking how to undo it.

1. You edited a file and want the last committed version back

The change is in the working directory only.

```
# terminal
$ git status --short
M monthly_revenue.sql

$ git restore monthly_revenue.sql

$ git status --short
```

The empty output after `git restore` is the point: the working directory matches the last commit again. `git restore <file>` overwrites the file on disk with the committed version. It is the one destructive undo in this list, because the edit was never recorded anywhere, so it is gone. Run `git`

`diff` first and look at what you are discarding.

2. You staged something you did not mean to

The change is in the staging area.

```
# terminal
$ git add -f secrets.txt

$ git status --short
A  secrets.txt

$ git restore --staged secrets.txt

$ git status --short
?? secrets.txt
```

How to read those two-letter codes, because this is where `--short` earns its place. The first column is the staging area, the second is the working directory. **A** means added to staging. **??** means untracked. So the file went from staged back to untracked, which is exactly what unstaging means: the file is still on disk, Git has stopped planning to commit it.

`git restore --staged <file>` moves a change out of the staging area and leaves the file alone. Nothing is lost. This is the safe undo, and it is the one you will use most.

(`git add -f` in the first line forces Git to add a file that `.gitignore` would normally skip. It exists for the rare deliberate case, and it is how a secret gets committed by someone who has been told once too often that Git is ignoring it.)

3. The last commit was wrong and you have not shared it

```
# terminal
$ git commit --amend -m "Ignore secrets, generated exports, and local noise"

$ git log --oneline
dbdb993 Ignore secrets, generated exports, and local noise
97d9490 Limit the revenue query to 2025 onward
e71cc3d Add monthly revenue query
```

--amend replaces the previous commit with a new one. The old commit is not edited; it is discarded and a new one takes its place, with a new hash. That is why the rule is absolute: **amend only a commit nobody else has**. Once a commit is pushed, somebody may have it, and replacing it makes their history and yours disagree in a way that takes an afternoon to sort out.

To add a forgotten file to the last commit rather than change the message: `git add the_file` then `git commit --amend --no-edit`, where `--no-edit` keeps the existing message.

4. A commit is already shared and is wrong

Now you cannot rewrite it. You add a new commit that undoes it.

```
# terminal
$ git revert --no-edit 97d9490
[main 1bed4f9] Revert "Limit the revenue query to 2025 onward"
Date: Mon Mar 2 10:02:00 2026 +0530
1 file changed, 1 deletion(-)

$ git log --oneline
1bed4f9 Revert "Limit the revenue query to 2025 onward"
dbdb993 Ignore secrets, generated exports, and local noise
97d9490 Limit the revenue query to 2025 onward
e71cc3d Add monthly revenue query
```

How it works:

- **git revert 97d9490** looks at what commit 97d9490 changed and creates a **new** commit doing the opposite. The original is still there, second from the bottom, which is the point: shared history is not edited, it is added to.
- **--no-edit** accepts the default message, `Revert "..."`, without opening an editor.
- The history now records both the change and the decision to undo it. Somebody reading it in six months can see that the 2025 filter was tried and reversed, which is more useful than a history where it never happened.

Where the change is	Command	Is anything lost?
Working directory only	<code>git restore <file></code>	yes , the edit is gone. Run <code>git diff</code> first
Staging area	<code>git restore --staged <file></code>	no, the file stays on disk
Last commit, not pushed	<code>git commit --amend</code>	the old commit is replaced. Safe if nobody has it
Any commit, already pushed	<code>git revert <hash></code>	no, a new commit undoes it and both remain

There is a fifth command, `git reset --hard`, which discards commits and working changes in one step. It is not in this table on purpose. Almost every situation where a beginner reaches for it is really situation 1, 2, or 4, and `--hard` is the one that loses work.

26.6 Branches, and why they are not an advanced topic

A **branch** is a movable label pointing at a commit. That is the whole definition, and it is why branches in Git are cheap enough to make casually.

The question: I want to try splitting the revenue query by product category, without breaking the version that currently works.

```
# terminal
$ git switch -c add-category-split
Switched to a new branch 'add-category-split'
```

Now change the file and commit on this branch. `main` still points at the working version; your new commits go onto `add-category-split`.

```
# terminal
$ git switch main
Switched to branch 'main'

$ git merge add-category-split
Updating 1bed4f9..6489661
Fast-forward
 monthly_revenue.sql | 5 +++--
 1 file changed, 3 insertions(+), 2 deletions(-)

$ git log --oneline
6489661 Split monthly revenue by product category
1bed4f9 Revert "Limit the revenue query to 2025 onward"
dbdb993 Ignore secrets, generated exports, and local noise
97d9490 Limit the revenue query to 2025 onward
e71cc3d Add monthly revenue query
```

How it works:

- **git switch -c add-category-split** creates a branch and moves onto it. **-c** means create; without it, **git switch** moves to a branch that already exists.
- **git switch main** moves back. Your files on disk change to match **main**. The category work is not lost; it is on the other branch.
- **git merge add-category-split** brings that branch's commits into **main**.
- **Fast-forward** means **main** had no commits of its own since the branch started, so Git moved the label forward. When both branches have moved, Git makes a **merge commit** instead, and if the same lines changed on both sides you get a **conflict**, which Git marks in the file for you to resolve by hand.

Why bother for solo work? Because it lets you stop halfway. A branch is a place to leave an unfinished idea without it sitting in your working directory blocking everything else. And because when you join a team, every change will go through one.

Naming. `add-category-split`, `fix-cancelled-orders`, `meera/monthly-report`. Short, specific, and readable in a list.

26.7 GitHub, pushing, and the pull request

Everything so far has been local. Your history lives in `.git` in your folder and nowhere else, which solves the "which version is current" problem but not the "my laptop died" problem or the "Imran is on leave" problem.

GitHub is a hosting service for Git repositories. A repository hosted there is called a **remote**. There are others, GitLab and Bitbucket among them, and they work the same way.

```
# terminal, after creating an empty repository on github.com
$ git remote add origin https://github.com/meera/riverstone-analysis.git

$ git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Writing objects: 100% (15/15), 1.42 KiB | 1.42 MiB/s, done.
To https://github.com/meera/riverstone-analysis.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

(The output above is what a first push prints. It is shown rather than run, because this chapter's examples run without a GitHub account.)

How it works:

- `git remote add origin <url>` records a nickname for the remote. `origin` is the conventional name for "the main place this repository lives". It is not a keyword; you could call it anything.
- `git push` sends commits the remote does not have.
- `-u origin main` sets the default, so that afterward plain `git push` and `git pull` know where to go. You need it once, on the first push.

Three commands cover the rest of normal use:

Command	What it does
<code>git clone <url></code>	copy an existing remote repository onto your machine, history and all
<code>git pull</code>	fetch what others have pushed and merge it into your branch
<code>git push</code>	send your commits to the remote

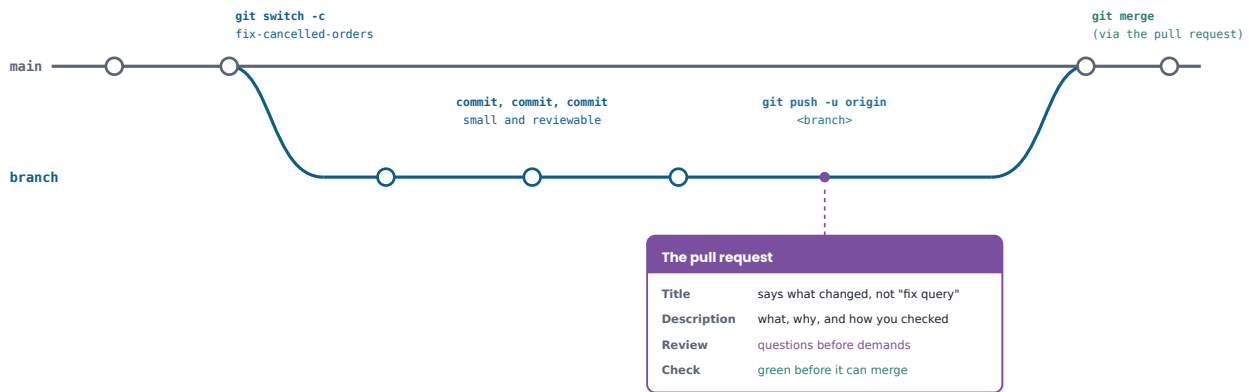
Pull before you push. If someone else pushed while you were working, your push is rejected, and the fix is `git pull` then push again. The rejection is Git preventing you from silently discarding their work.

The pull request

On a team, nobody pushes straight to `main`. The flow is the same four steps everywhere:

From a branch to merged: the loop you will run every working day

The unit of work on a team is not a commit. It is a pull request.



Nothing here is optional at work. Even a one-line change goes round this loop, which is why the loop has to be cheap.

Figure 26.3 – The unit of work on a team is not a commit. It is a pull request.

1. **Branch.** `git switch -c fix-cancelled-orders`
2. **Commit and push.** Your work goes to the remote on that branch.
3. **Open a pull request.** On GitHub, propose merging your branch into `main`. A pull request is a request for review, a place for the conversation, and a record of why the change was made.
4. **Review, then merge.** Somebody reads it, comments, you change things, and it merges.

What a good pull request description contains, and this matters more for data work than for most code: what changed, **why**, and how you checked it. "Fixed the revenue query" is not reviewable. "Cancelled orders were being counted in monthly revenue. Excluded them. January drops from ₹115,400 to ₹104,210, which now matches the ERP's own figure" is a description a reviewer can actually verify.

What to look for when reviewing someone else's query, which you will be asked to do:

- Does it answer the question in the description?
- Is the **grain** right? One row per what?
- Are cancelled, test, and internal records handled, or silently included?
- Do the numbers reconcile with something known?
- Would you be able to change this in six months?

26.8 One automated check on every push

A repository can run a check automatically whenever someone pushes. On GitHub this is called **GitHub Actions**, and the smallest useful version is worth the twenty minutes it takes to set up: it catches the broken thing before a reviewer wastes time on it.

Create a file at `.github/workflows/checks.yml`:

```

name: checks
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python -m pytest -q

```

How it works, line by line:

- **name: checks** is what appears on the pull request. Any name will do.
- **on: [push, pull_request]** is the trigger. This runs on every push and on every pull request.
- **jobs: then test:** declares one job named `test`. A workflow can have several, running in parallel.
- **runs-on: ubuntu-latest** asks GitHub for a fresh Linux machine. It is created for this run and destroyed afterward, which is why the next steps have to install everything.
- **steps:** run in order. A step is either a prepared action (`uses:`) or a command (`run:`).
- **uses: actions/checkout@v4** copies your repository onto that machine. Without it the machine is empty. The `@v4` pins the version, which is the difference between a workflow that keeps working and one that breaks in a year.
- **actions/setup-python@v5 with python-version: "3.12"** installs that Python. Quote the version: unquoted, YAML reads `3.10` as the number 3.1.
- **run: pip install -r requirements.txt** installs your dependencies on the fresh machine.
- **run: python -m pytest -q** runs your tests. `-q` is quiet mode, which prints a dot per test instead of a line. If this command exits with an error, the check fails and the pull request shows a red cross.

Setting	What it means in plain words	Value used here	What happens if you change it
<code>on:</code>	when the workflow runs	<code>[push, pull_request]</code>	narrow it to <code>[pull_request]</code> and pushes to your own branch stop burning minutes; add <code>schedule:</code> to run it nightly
<code>runs-on:</code>	which machine	<code>ubuntu-latest</code>	<code>windows-latest</code> or <code>macos-latest</code> cost more minutes. Use Linux unless you need otherwise
<code>python-version:</code>	which Python	<code>"3.12"</code>	set it to the version you actually develop on, or the check passes here and fails on your machine
<code>@v4</code> , <code>@v5</code>	which version of the prepared action	pinned	leave the version off and you get the latest, which changes under you without warning

What to put in this check for a data repository, in order of value: does the Python import and run, do the tests pass, does a linter accept the SQL, and does a small query against a test database still return the expected number of rows. Chapter 47 turns that last one into proper data-quality testing, and Chapter 52 takes the whole workflow further into deployment.

26.9 A repository a stranger can run

The test: hand the URL to a colleague who has never seen it, say nothing, and see whether they get an answer within fifteen minutes. Everything in this section exists to pass that test.

Structure

```
riverstone-analysis/
├── README.md
├── .gitignore
├── .env.example
├── requirements.txt
├── sql/
│   ├── patterns/                ← the Chapter 13 pattern library, one file per pattern
│   │   ├── top_n_per_group.sql
│   │   ├── running_total.sql
│   │   └── gaps_and_islands.sql
│   └── reports/
│       └── monthly_revenue.sql
├── scripts/
│   └── monthly_report.py        ← Chapter 20
├── notebooks/
│   └── 2026-02-churn-exploration.ipynb
├── docs/
│   └── metrics.md              ← the definitions from Chapter 23
```

Two things to copy. **One file per SQL pattern**, each with a comment at the top saying what question it answers, because a pattern library you cannot search is a file you rewrite. And **dates at the front of exploratory notebooks**, `2026-02-...`, so they sort in time order, the habit from Chapter 2.

Files Git cannot diff

Git stores any file you give it, but it can only *show you the change* in a text file. A `.sql` file, a `.py` file, a `.md` file, a `.yaml` file: all text, all diffable. An `.xlsx` workbook, a `.pbix` Power BI file, a `.docx`, a PNG: all binary. Git will happily version them, and every commit stores a whole new copy, so the repository grows and `git diff` tells you only "Binary files differ".

Three workable answers, in order of preference:

File	What to do
Power BI	save in the .pbip project format instead of <code>.pbix</code> . It writes the model and the report as text files, which diff properly. Chapter 16 introduced it
Excel macros and VBA	export the modules (<code>.bas</code> , <code>.cls</code> , <code>.frm</code>) into a <code>vba/</code> folder and commit those, the habit from Chapter 19. The workbook itself stays out, or goes in as a single dated copy
A workbook or document that must be versioned as a whole	commit it, but accept that history is a chain of copies. Keep a <code>CHANGELOG.md</code> beside it saying what changed in each version, because Git cannot tell you

And the blunt one: **data files do not belong in a repository at all**. A CSV export is regenerable, often large, and sometimes confidential. Commit the query that produces it.

The README

Six headings. This is the whole thing, and it takes twenty minutes:

Heading	What goes in it
What this is	one paragraph. What question this repository answers, for whom
What you need	Python version, database, any account. Be specific: "PostgreSQL 16" not "a database"
Setup	the exact commands, in order, that take someone from clone to running. Copy them from a terminal where you actually ran them
How to run it	the command for the main thing, with an example of the output
How the data is defined	the grain, the filters, and the metric definitions, or a link to <code>docs/metrics.md</code>
Who to ask	a name. A repository with no owner is a repository nobody maintains

The test for the setup section: delete the folder, clone it fresh, and follow your own instructions without using anything you remember. Everybody's instructions are missing a step. This is how you find which one.

Documentation that survives

Most documentation fails for one of three reasons, and each has a fix that costs nothing:

Why it fails	Fix
It lives somewhere nobody looks	put it next to the thing it describes: the README in the repository, the definition in the model, the comment in the query
It goes stale and nobody trusts it	date it, and write the parts least likely to change. A metric definition lasts; a screenshot does not
It explains what rather than why	the code already says what. Write down why cancelled orders are excluded and who decided

For a data team the highest-value documentation is not a wiki. It is three things: the **metric definitions** from Chapter 23, the **business rules** you wrote as a BA in Chapter 25, and a one-line comment at the top of every query saying which question it answers.

26.10 How the work is actually planned

You will join a team that runs some version of Agile. Here is enough to be useful in your first week.

Agile is a preference, not a process: working in short cycles, showing something real at the end of each one, and changing the plan when what you learn says the plan was wrong. Two ways of organizing it dominate.

Scrum runs in fixed cycles called **sprints**, usually two weeks.

Ceremony	When	What happens
Sprint planning	first day	the team picks what it will finish in this sprint from the backlog
Daily standup	every morning, 15 minutes	each person: what I did, what I am doing, what is blocking me
Sprint review	last day	show the working thing to the people who asked for it
Retrospective	last day	what to change about how we work, not about what we built

Kanban has no sprints. Work flows across a board continuously, and the discipline is a **work-in-progress limit**: no more than n items in the "in progress" column at once, so that finishing beats starting.

Which suits data work. Analysis requests arrive unpredictably and vary enormously in size, which fits Kanban. Building a pipeline or a dashboard is a project with a shape, which fits Scrum. Most data teams end up with a mix: a board with WIP limits for requests, and sprint-like cycles for the bigger builds.

Reading a Jira board. Jira is the tool most companies use for this. Four things to know:

- An **issue** is one unit of work, with a key like `DATA-118`. That key belongs in your branch name and your pull request title, so a year later somebody can trace the code back to the request.
- **Epic, story, task, bug** are sizes and kinds. A story is a user-visible piece of work, the Chapter 25 shape; a task is a piece of it; an epic groups many.
- **Story points** estimate relative size, not hours. A 5 is bigger than a 3. Teams argue about this more than it deserves.
- The **backlog** is everything not yet started, in priority order. If your work is not in it, it does not exist to the people planning.

The honest part. Every one of these can become theatre: standups that are status reports to a manager, retrospectives where nothing changes, estimates treated as commitments. The practices are worth what the team makes of them. What is worth defending, whatever the process is called, is the short cycle and the working thing at the end of it.

26.11 Working with an AI assistant

You will use one. Most data teams already do. The skill that matters is not prompting. It is knowing what to hand over, and how to check what comes back.

What to delegate, and what not to

Delegate	Keep
boilerplate you have written before: a connection block, argument parsing, a chart's formatting	deciding what the question is
translating between dialects: this PostgreSQL query in MySQL	choosing the grain and the filters
explaining an error message or an unfamiliar function	judging whether a number is plausible

Delegate	Keep
a first draft of a docstring, a README section, a regular expression	anything where being wrong is expensive and the error would be invisible
generating test data or edge cases you had not thought of	the final read of the numbers before they go to somebody

The pattern: **delegate the typing, keep the judgment**. An assistant is good at things with a known right answer and poor at things that depend on context it cannot see, which for data work is most of what matters.

How to check what it gives you

The failure mode is not nonsense. Nonsense is quick to spot. The failure mode is code that looks right, runs, and returns a number that is wrong.

1. **Run it on data where you know the answer.** Chapter 12's mini database has twelve orders. If a generated query cannot get January right on twelve rows, it will not get it right on two hundred thousand.
2. **Read it line by line before running it.** If you cannot explain every line, you cannot maintain it, and you certainly cannot defend the number it produced. This is the same standard this book applies to its own code.
3. **Check the joins and the filters specifically.** These are where generated SQL goes wrong: a join that fans out and doubles the revenue, a `WHERE` clause that quietly drops NULLs, a missing exclusion of cancelled orders.
4. **Reconcile.** Compare the total against something already known, exactly as Chapter 25 section 10 does for a data product.
5. **Never paste an error you do not understand back and accept the next answer.** Two rounds of that and you have code nobody understands, including the assistant.

What must never be pasted into one

This is a rule, not a preference, and breaking it is a disciplinary matter in many companies.

- **Real customer data.** Names, contact details, addresses, anything identifying a person.
- **Credentials.** Passwords, keys, tokens, connection strings.
- **Anything covered by a contract or a law.** Under India's Digital Personal Data Protection Act 2023, and under the GDPR where it applies, personal data sent to a third-party service is a disclosure, and it is your employer's obligation, not the tool's. Chapter 64 covers the governance side in full.
- **Unreleased financials, or anything a competitor would like.**

The safe habit: **work on the schema, not the rows**. Paste the table structure, the column names, and a made-up row or two. An assistant helping you write a query has no use for real data, and Riverstone's own data in this book is invented for exactly this reason.

If your company has an approved assistant with a data-processing agreement, the rules may be looser. Find out what they are before you need them, not afterward.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Committing a secret	a password or key appears in <code>git log -p</code> , and now it is in everyone's clone	<code>git rm --cached</code> , commit, and then change the password . Removing the file does not remove the history
Commit messages that say "update"	six months later nobody can find when a rule changed	one line saying what changed and, where it is not obvious, why
One giant commit on Friday	the history is useless for finding a mistake, because every change is in the same snapshot	commit each finished piece of work as you finish it
Committing generated files	every <code>git diff</code> is full of noise; the repository is large	put exports, <code>__pycache__</code> , and built workbooks in <code>.gitignore</code>
Committing a notebook with its outputs	huge diffs over a rerun that changed nothing but a timestamp	clear outputs before committing, or use a tool that strips them
Working only on <code>main</code>	a half-finished change is sitting in the one branch people use	a branch per piece of work, however small
Copying a <code>git reset --hard</code> from a forum post	uncommitted work is gone, with no undo	decide which of the four undos you are in first. <code>restore</code> , <code>restore --staged</code> , <code>--amend</code> , <code>revert</code> cover almost everything
Amending or force-pushing a shared commit	a colleague's <code>git pull</code> fails, or silently loses work	amend only what you have not pushed. Shared history is undone with <code>revert</code>
Not pulling before starting	a conflict every time, on work you did not need to redo	<code>git pull</code> before you branch, and before you push
Branch names like <code>test2</code>	nobody, including you, can tell what is in them	<code>fix/duplicate-invoice-rows</code> , or the issue key: <code>DATA-118-late-file-handling</code>
A repository with no README	the work is technically shared and practically private	six headings, twenty minutes, today
Setup instructions written from memory	a colleague cannot get past step 3	clone it fresh into a new folder and follow your own instructions
Absolute paths in scripts	it runs on your laptop and nowhere else	paths relative to the repository, and configuration in <code>.env</code>
A failing check on <code>main</code> that nobody fixes	the red cross stops meaning anything and the check is ignored	fix it or delete it. A check people ignore is worse than no check
Pasting real customer data into an AI assistant	a disclosure you cannot take back, and possibly a legal one	work on the schema and made-up rows, never the real ones
Accepting generated SQL because it runs	a number that is wrong and looks right	read every line, check the joins and filters, reconcile against something known

In the real world: the rule nobody could date

In February 2026 Meera Iyer was asked a question she could not answer.

Vikram Singh, comparing the January sales report against his own notes, found the revenue figure lower than he expected by a little under two lakh rupees. Meera checked. The report excluded cancelled orders. That was correct, and it was what the finance team wanted. But Vikram's copy of the December report, sitting in his inbox, had included them.

So the rule had changed somewhere between December and January. The question he asked was reasonable and she could not answer it: **when did it change, and who decided?**

The script lived in a folder on her laptop, next to `monthly_report_old.py` and `monthly_report_v2_working.py`. The file dates told her only when she had last saved each one. There was no record of what she had changed, and no record of the conversation that caused it. She had made the change, she was fairly sure, after a conversation with the finance assistant in early January. She could not prove it, and she could not prove anything else either.

The report was re-run three times that week while two teams argued about which number was right. The rule turned out to be correct all along. What cost the week was not the rule. It was that nothing recorded the decision.

The repository she set up the following Monday was small and unglamorous: four SQL files, two Python scripts, a `.gitignore`, and a README with six headings. The habit that mattered was the one that felt like overhead at the time. Every commit message that changed a business rule said who asked for it:

```
# terminal
$ git log --oneline -n 3
837e258 Exclude cancelled orders from monthly revenue (agreed with finance, 8 Jan)
6489661 Split monthly revenue by product category
1bed4f9 Revert "Limit the revenue query to 2025 onward"
```

Six weeks later the same question came up about a different rule, from a different person. It took forty seconds.

There was one other thing she got wrong first, which is worth saying because almost everyone does it once. Her first push included the `.env` file with the database password in it. Farah spotted it while looking at the repository on GitHub. Meera deleted the file and committed the deletion, which felt like a fix and was not: the password was still in the history, visible to anyone with access to the repository, and it stayed there. The real fix took ten minutes with the IT contractor: **change the password**. Then add `.env` to `.gitignore`, then commit `.env.example` so the next person knows what to supply.

The lesson is not that Git is complicated. It is that a repository remembers everything, which is exactly why it is useful and exactly why a secret in it is not a mistake you can quietly undo.

Tools

Tool	What it is for	Notes
Git	the version control system itself	Free, runs locally, works with no internet. Chapter 6 installed it
GitHub	the hosting service most teams and most portfolios use	Free accounts include unlimited public and private repositories. This is where a portfolio lives
GitLab, Bitbucket, Azure Repos	the same job at companies that do not use GitHub	The commands are identical. Only the website changes
The Git panel in VS Code	staging, committing, and branching without the terminal	Useful. Learn the commands first, so that you can read what it is doing and fix it when it stops working
GitHub Desktop, Sourcetree, Fork	graphical Git clients	Same caveat. A graphical client is a convenience, not a substitute for the model in section 26.1
GitHub Actions	running checks automatically on every push	Free for public repositories, with a monthly allowance for private ones. Chapter 52 goes further
pre-commit	running checks <i>before</i> a commit is created, on your machine	Catches the formatting and the stray secret before they reach the history
gitleaks, git-secrets	scanning a repository for credentials	Worth adding to any repository that touches a database
nbstripout	clearing notebook outputs automatically on commit	Solves the single worst diff problem in data work
Jira, Azure Boards, Linear, Trello	the board the work is planned on	You will not choose this. Learn to read whichever one you are given
Markdown	what README files and most documentation are written in	Plain text with a few symbols. An hour to learn, used for the rest of your career
MkDocs, Docusaurus, GitHub Pages	turning a folder of Markdown into a browsable site	Useful once the documentation outgrows one README
Confluence, Notion, SharePoint	company wikis	Fine for decisions and context. Poor for anything that must match the code, because it drifts
AI coding assistants	drafting, translating, and explaining code	Section 26.11. Delegate the typing, keep the judgment

The project: one repository for everything you have built

By now Part II has produced a folder of work. This project turns it into one repository a stranger could run, and it is the raw material for Chapter 27's portfolio. Budget two hours.

1. Create the repository. A new folder, `git init`, and a first commit containing only a `.gitignore` and an empty `README.md`. Starting with `.gitignore` is deliberate: the rules exist before anything can be committed by accident.

2. Move the work in, organized. Use the structure from section 26.9: `sql/patterns/` , `sql/reports/` , `scripts/` , `notebooks/` , `docs/` . At minimum, bring in the pattern library from Chapter 13 (one file per pattern, each with a comment saying what question it answers), the report script from Chapter 20, and the metric definitions from Chapter 23 as `docs/metrics.md` . Commit in several commits, grouped by what they are, not all at once.

3. Make the secrets safe. Anything holding a password becomes `os.environ[...]` . Write a `.env.example` with the names and no values. Run `git check-ignore -v .env` and confirm it is being ignored by the rule you think it is.

4. Write the README. Six headings, from the table in section 26.9. Then apply the test: clone the repository into a new folder somewhere else, and follow your own setup instructions without using anything you remember. Fix what is missing. Commit the fix.

5. Do one full cycle on a branch. Pick one real improvement, however small: a filter that should have been there, a comment at the top of a query, a metric definition that is vague. Branch, change, commit, push, open a pull request against your own `main` , write the description as though a reviewer who has not seen the change will read it, then merge it. Read your own diff before you merge. This is the loop you will run every working day for the rest of your career.

6. Add one check. The workflow from section 26.8. Push a deliberate failure and watch it go red, then fix it and watch it go green. A check you have never seen fail is a check you do not know works.

Stretch. Review someone else's repository, from a study group or from a public one, and write three comments: one question about something you did not understand, one thing you would do differently and why, and one thing that was well done. Reviewing badly is the most common way new team members make a poor first impression, and it is entirely avoidable.

What "done" looks like: a URL you could put on a CV, where a stranger can read what the work is, see how it was built, and run it.

You've got it when...

- You can name the three places a file lives in Git and say which one a given error message is talking about.
- You commit your work in pieces that make sense, with messages that would help a colleague, or you, in a year.
- Your `.gitignore` exists before your first commit, and you know why `git rm --cached` is needed when it does not.
- You know that deleting a committed secret does not remove it, and that the fix is to change the credential.
- Given something that went wrong, you can say which of the four undos applies before you type a command.
- You work on a branch without being told to, and your branch names say what is in them.

- You can open a pull request whose description a reviewer can act on, and review someone else's without either rubber-stamping it or being unpleasant.
 - You can read a `git diff` and say what changed, line by line.
 - You have one repository whose README lets a stranger run your work in fifteen minutes.
 - You can read a Jira board and say what the team is doing this sprint.
 - You use an AI assistant for the typing and not for the judgment, and you can list what must never be pasted into one.
-

Recap

Git keeps a file in three places: the working directory where you edit, the staging area where you choose what belongs in the next snapshot, and the history where snapshots are permanent. Nearly every confusing Git message becomes clear once you ask which of the three it means.

The daily loop is small: `git status` to see where you are, `git add` to choose, `git commit -m` to record, `git log` to read. A commit message says what changed and, when the code cannot show it, why. That second half is the part that pays back, because the code can always tell you what.

Secrets and generated files stay out, through a `.gitignore` written before the first commit. Code reads credentials from the environment, and a committed `.env.example` tells the next person what to supply. A secret that reaches the history is not fixed by deleting the file. It is fixed by changing the credential.

There are four undos, and choosing between them is the whole skill: `git restore` for an edit you have not staged, `git restore --staged` for something staged by mistake, `git commit --amend` for the last commit if you have not shared it, and `git revert` for anything already shared. Branches are not advanced; they are how you try something without risking what works.

GitHub adds the shared copy, the pull request, and the review. A good pull request is small and has a description that tells the reviewer what to look for. A good review asks questions before it makes demands. One automated check on every push turns a promise into a fact, and checks that people ignore are worse than no checks.

A repository a stranger can run needs structure, a six-heading README, and setup instructions you have tested by following them in a fresh clone. Documentation survives when it lives next to what it describes, carries a date, and records why rather than what. Files Git cannot diff, such as Power BI and Excel workbooks, need a text-based format or exported modules instead.

Work is planned in short cycles. Scrum has sprints and four ceremonies; Kanban has a continuous flow and a work-in-progress limit; most data teams mix them. Read the board you are given, put the issue key in your branch name, and remember that the practices are worth exactly what the team makes of them.

You will work with an AI assistant. Delegate the typing: boilerplate, translation between dialects, first drafts, explanations. Keep the judgment: the question, the grain, the filters, and the final read of the numbers. Check what comes back on data where you know the answer, read every line before you run

it, look hardest at joins and filters, and reconcile. Never paste real customer data, credentials, or anything covered by a contract or a law. Work on the schema, not the rows.

Practice exercises

Warm-up

1. Name the three places a file lives in Git, and say what `git add` and `git commit` each move it between.
2. You run `git status` and see a file listed under "Changes not staged for commit". What has happened to it, and what has not?
3. Write three commit messages for these changes, each one line: (a) you added a filter excluding cancelled orders, after finance asked for it; (b) you renamed a column in a query from `amt` to `order_value`; (c) you fixed a join that was double-counting orders with more than one line.
4. Which of these belong in `.gitignore`, and why: `monthly_report.py`, `.env`, `exports/january.csv`, `README.md`, `__pycache__`, `sales_data.xlsx`, `requirements.txt`?

Core

5. For each situation, say which of the four undos you would use and why: (a) you edited a query, made it worse, and have not staged anything; (b) you ran `git add .` and it picked up a 40 MB export; (c) you committed with the message "asdf" two minutes ago and have not pushed; (d) you pushed a commit on Tuesday that dropped the date filter, and two colleagues have pulled since.
6. **Predict before running.** You have committed a file, then edited it, then run `git add` on it, then edited it again. Write down what you expect `git status` to show for that one file, and how many entries it will have. Then do it and check.
7. Write a `.gitignore` for a repository containing Python scripts, SQL files, a Jupyter notebook, a `.env`, and an `exports/` folder that fills with `.xlsx` and `.csv` files. Comment each line.
8. You are reviewing a colleague's pull request. It changes a query's `JOIN` from `LEFT` to `INNER` and the title is "fix query". Write three comments you would leave, in the order you would leave them.
9. A generated query gives you last quarter's revenue as ₹94,32,000. List four specific checks you would run before putting that number in an email, in the order you would run them.
10. Write a README "What you need" and "Setup" section for the Chapter 20 report script, assuming the reader has a laptop and nothing else. Be specific enough that a version number appears at least twice.

Stretch

11. Section 26.8's workflow runs on every push. Describe two changes you would make to it for a repository where the tests take twenty minutes and most pushes are to branches nobody is reviewing yet, and say what each change costs you.
12. Your team's board has fourteen items in the "in progress" column and three people. Write what you would say in the retrospective, and what you would propose changing. Name the Kanban idea

you are appealing to.

- l3. A colleague pastes a production customer table into an AI assistant to get help writing a query, and the output is good. Write what you would say to them, and then write the policy line you would propose for the team, in one sentence each.

Think about it (no calculation needed)

- l4. The chapter says a commit message should record **why** rather than what. But the "why" usually lives in a conversation with someone outside the team, which is not in the repository. What practical habits close that gap, and what does that suggest about where the issue key belongs?
- l5. An AI assistant will write a correct-looking query faster than you can write a wrong one. If that is true, what exactly is the analyst being paid for, and how should that change what you spend your learning time on?

Key terms

version control · repository · working directory · staging area · commit · commit hash · `git init` · `git status` · `git add` · `git commit` · `git log` · `git diff` · untracked file · tracked file · `.gitignore` · `git check-ignore` · `git rm --cached` · `.env` · `.env.example` · environment variable · `git restore` · `git restore --staged` · `git commit --amend` · `git revert` · branch · `git switch` · `git merge` · fast-forward · remote · `git push` · `git pull` · clone · pull request · code review · merge conflict · continuous integration · workflow · README · Markdown · Agile · sprint · Scrum · standup · retrospective · backlog · Kanban · work-in-progress limit · issue key · story point · AI assistant

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 27, the capstone**, turns this repository into a portfolio that a hiring manager will actually open.
- **Chapter 29, Python as Software**, adds the things a repository like this grows into: a package layout, tests, and type hints, all of which the check in section 26.8 will then run for you.
- **Chapter 32, dbt**, puts your SQL transformations under the same discipline, with version control and automated tests as the default rather than an addition.
- **Chapter 34, the command line**, goes properly into the terminal that this chapter used without explaining: paths, environment variables, and what a shell actually does.
- **Chapter 44** builds an end-to-end data science project, which is a repository with the same requirements as this one and more moving parts.
- **Chapter 47** turns the "check on every push" idea into data quality tests and contracts that run against the data rather than the code.

- **Chapter 52** takes continuous integration to deployment: containers, cloud, and infrastructure defined as text files in a repository.
- **Chapter 56, MLOps**, versions models and the data they were trained on, because a model with no record of what produced it cannot be debugged.
- **Chapter 64** covers the governance side of what must never leave the building, including what an AI assistant counts as.
- **Chapter 82** includes take-home assignments, which are graded on exactly the repository standards in section 26.9.

Answers to practice exercises

1. The **working directory** (the files as you see them), the **staging area** (what you have selected for the next commit), and the **history** (commits, permanent). `git add` moves a change from the working directory to the staging area. `git commit` moves everything staged into the history, as one snapshot with a message.
2. The file has been **edited in the working directory** and those edits have **not been staged**. Git knows the file, is tracking it, and can see it differs from the last commit. Nothing about it will be in the next commit unless you run `git add` on it first.
3. (a) Exclude cancelled orders from monthly revenue (agreed with finance, 8 Jan) : the why is the half that matters, and the date and the requester make it findable. (b) Rename `amt` to `order_value` in the revenue query , which is self-explanatory, so no why is needed. (c) Fix duplicate rows from `order_items` join in revenue query , which names the cause rather than only the symptom, so the next person recognizes the same bug elsewhere.
4. In `.gitignore` : `.env` (a secret), `exports/january.csv` (generated), `__pycache__` / (generated), and `sales_data.xlsx` (data, and probably regenerable; if it is source data that cannot be regenerated, it still usually belongs somewhere other than a repository). Committed: `monthly_report.py` , `README.md` , `requirements.txt` , all of them text, and all needed to run or understand the work.
5. (a) `git restore <file>` : the change is unstaged, so restoring from the last commit costs nothing but the bad edit. (b) `git restore --staged <file>` : unstage it; the file itself is untouched, and this is also the moment to add it to `.gitignore` . (c) `git commit --amend -m "..."` : the commit is the last one and has not been shared, so rewriting it is safe. (d) `git revert <hash>` : it is shared, so the history must not be rewritten. `revert` adds a new commit that undoes it, and everyone's `git pull` works normally.
6. `git status` shows the file **twice**: once under "Changes to be committed" (the first edit, staged) and once under "Changes not staged for commit" (the second edit, not staged).

```
# terminal
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   monthly_revenue.sql

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   monthly_revenue.sql
```

In the short form it is one line with two status letters, **MM**: modified in the staging area, and modified again in the working directory.

```
# terminal
$ git status --short
MM monthly_revenue.sql
```

This is the clearest single demonstration that the staging area is a real, separate place: Git is holding two different versions of one file, and only the staged one would go into the commit.

7.

```
# .gitignore
# Secrets
.env

# Generated exports, whatever the format
exports/

# Belt and braces: spreadsheets and CSVs anywhere in the repository
*.xlsx
*.csv

# Python and notebook noise
__pycache__/
.ipynb_checkpoints/
```

The `exports/` rule alone would cover the files in that folder; the `*.xlsx` and `*.csv` lines also catch the copy somebody inevitably saves in the wrong place. Nothing here excludes `.sql` or `.py` or the notebook itself, which are the work.

8. (1) "What was the reason for the join change? If some orders have no line items, this will now drop them from the report, and I want to check that is intended." The question comes first, because you may be wrong. (2) "Could the title say what the fix is? Something like 'Exclude orders with no line items from revenue'. It will matter when someone searches the history." (3) "Is there a case in the January data where this changes the total? If so, that number is worth putting in the description so the reviewer can check it." Ordering matters: question, then the cheap fix, then the request for evidence.

9. (1) **Read every line** and say what each does, particularly the joins and the `WHERE` clause. (2) **Run it on the twelve-row mini database** from Chapter 12, where you can count the answer by hand. (3) **Check the grain**: does one row mean one order or one order line, and is the join fanning out? (4)

Reconcile the total against something already trusted, such as the ERP figure or last quarter's report, and explain any difference before sending it.

10. A model answer:

What you need. Python 3.11 or later · PostgreSQL 16, with the `riverstone` database loaded (see `sql/setup/`) · the packages in `requirements.txt` · a `.env` file with `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, copied from `.env.example`.

Setup.

1. `git clone <url> && cd riverstone-analysis`
2. `python -m venv .venv && source .venv/bin/activate` (on Windows, `.venv\Scripts\activate`)
3. `python -m pip install -r requirements.txt`
4. `cp .env.example .env` and fill in your database password
5. `python scripts/monthly_report.py --month 2026-01`

Two version numbers appear because "a recent Python" is not a setup instruction. The test of this section is not whether it reads well; it is whether it works in a fresh clone.

11. Two reasonable changes. (1) **Run the full suite only on pull requests and on pushes to main**, and run a faster subset on other branches. The cost is that a broken branch may stay broken for longer, until the pull request is opened. (2) **Cache the dependency installation** between runs, which usually takes the largest fixed cost out of a twenty-minute job without changing what is tested. The cost is a small risk of a stale cache hiding a dependency problem, which is why the cache key includes `requirements.txt`. A third answer, running the slowest tests on a schedule overnight instead of on every push, is also defensible: it trades immediate feedback for throughput, and it is only safe if somebody actually reads the overnight result.

12. "We have fourteen things in progress and three people, so on average each of us is holding four or five open items and finishing none of them quickly. Everything is started and nothing is done, which is why the requesters think we are slow even though we are busy. I would propose a **work-in-progress limit** of two per person: nothing new starts until something moves out of the column. We should expect the first week to feel worse, because some of us will have nothing to start, which is the point: the right response to a blocked item is to help finish someone else's, not to pick up a fifteenth."

13. To the colleague: "That table has real customer names in it, so pasting it counts as a disclosure under our data protection obligations, whatever the tool's own policy says. It is not about whether the answer was good. Use the schema and a couple of made-up rows instead; you will get the same query." The policy line: "Schema, column names, and invented sample rows may be shared with approved AI assistants; real personal data, credentials, and unreleased financials may not, in any tool, including approved ones."

14. The gap is real: the reason for a rule usually lives in a meeting, an email, or a conversation at someone's desk. Three habits close it cheaply. **Put the issue key in the branch name and the pull request title**, so the commit leads back to the ticket, which leads back to the request and the discussion on it. **Write the requester and the date into the commit message** when a business rule

changes, as Meera did. And **keep the rule itself in docs/metrics.md**, dated, so the current answer is findable without reading history at all. The deeper point is that the repository is one of several records and does not have to hold everything. It has to hold the **link** to wherever the decision lives, because links survive people leaving and memories do not.

15. The analyst is paid for the part that cannot be delegated: knowing which question is worth asking, knowing what the data means and where it lies, and being accountable for a number that someone will act on. Speed of typing was never the scarce thing; it only looked like the job because it took most of the hours. For learning time, that argues for spending less of it memorizing syntax you can now generate, and more on the three things an assistant cannot do for you: **domain knowledge** (what a cancelled order means at this company), **verification** (how to tell a plausible number from a right one), and **judgment about the question** (Chapters 5, 23, and 25). It also argues for keeping the ability to read code fluently, since checking is now a larger share of the work than writing, and you cannot check what you cannot read.
